Digital Systems

Lecture 7

Memory and Addressing

Mark van der Wilk

Hilary Term 2025

Department of
COMPUTER
SCIENCE

Following the course of past years by Mike Spivey

Questions about the course so far?

? How can we access memory?

We have seen `push` and `pop`, but they were designed for the stack!

? How can we place values in a subroutine in memory?

I.e. *local* variables.

? How can place values in memory that can be accessed in all subroutines?

I.e. *global* variables.

? How can we handle variable amounts of data?

I.e., what happens if we don't know when we write the program, how many variables we need to store?

?

How can we access memory?

Accessing Memory

Instructions for storing and loading locations in memory: `str` and `ldr`.

-
-
-

Accessing Memory

Instructions for storing and loading locations in memory: `str` and `ldr`.

- Remember: Memory is accessed by providing an *address*.
-
-

Accessing Memory

Instructions for storing and loading locations in memory: `str` and `ldr`.

- Remember: Memory is accessed by providing an *address*.
- `ldr` and `str` have different ways of calculating the address.
-

Accessing Memory

Instructions for storing and loading locations in memory: `str` and `ldr`.

- Remember: Memory is accessed by providing an *address*.
- `ldr` and `str` have different ways of calculating the address.
- These are called *addressing modes*.

Accessing Memory

Instructions for storing and loading locations in memory: `str` and `ldr`.

- Remember: Memory is accessed by providing an *address*.
- `ldr` and `str` have different ways of calculating the address.
- These are called *addressing modes*.

We can use the `ldr` and `str` instructions in two different syntaxes, to use the different addressing modes:

```
xxr r0, [r1, #20]      @ Load/store r0 in r1 + 20
xxr r0, [r1, r2]       @ Load/store r0 in r1 + r2
```

Accessing Memory: Peculiarities

- Typical encoding: Only the lower registers `r0` - `r7` can be used.
-
-
-

Accessing Memory: Peculiarities

- Typical encoding: Only the lower registers `r0` - `r7` can be used.
- Alternative encoding for:

```
xxr r0, [sp, #20]
xxr r0, [pc, #40]
```

-
-

Accessing Memory: Peculiarities

- Typical encoding: Only the lower registers `r0` - `r7` can be used.
- Alternative encoding for:

```
xxr r0, [sp, #20]
xxr r0, [pc, #40]
```

- Different constant ranges allowed.
-

Accessing Memory: Peculiarities

- Typical encoding: Only the lower registers `r0` - `r7` can be used.
- Alternative encoding for:

```
xxr r0, [sp, #20]
xxr r0, [pc, #40]
```

- Different constant ranges allowed.
- Addresses must be *aligned*, based on their size.
  E.g. for 4-byte quantities, we need `addr % 4 == 0`.

Accessing Memory: Peculiarities

- Typical encoding: Only the lower registers `r0` - `r7` can be used.
- Alternative encoding for:

```
xxr r0, [sp, #20]
xxr r0, [pc, #40]
```

- Different constant ranges allowed.
- Addresses must be *aligned*, based on their size.
  E.g. for 4-byte quantities, we need `addr % 4 == 0`.

Accessing Memory: Peculiarities

- Typical encoding: Only the lower registers `r0` - `r7` can be used.
- Alternative encoding for:

```
xxr r0, [sp, #20]
xxr r0, [pc, #40]
```

- Different constant ranges allowed.
- Addresses must be *aligned*, based on their size.
  E.g. for 4-byte quantities, we need `addr % 4 == 0`.

In Native ARM code, we also allow

```
ldr r0, [r1, r2, LSL #x]      @ r1 + r2 << x
```

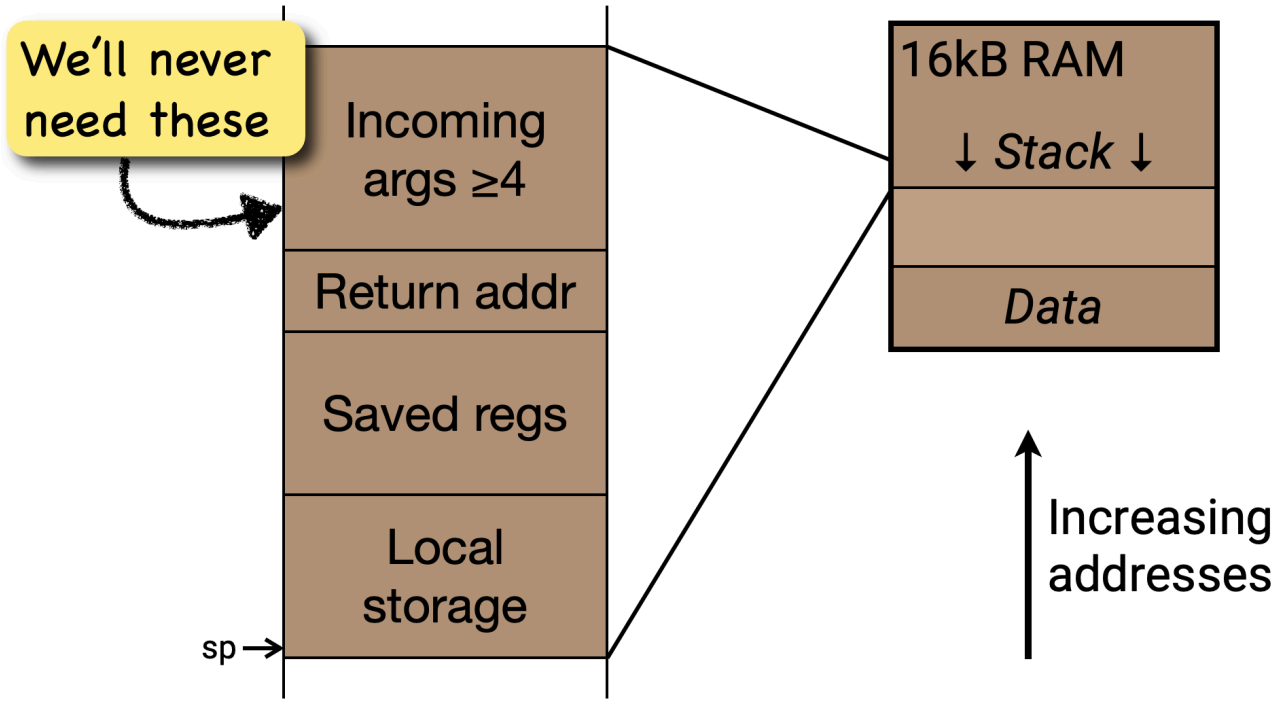
to make this easy to enforce. In Thumb, we made need more instructions.

?

How can we store (in memory) local variables?

Local Variables

Local variables are allocated on the stack. Remember why?



Factorial Example: Allocating Local Variables

Instead of restoring `r4`, `r5`, let's keep variables in memory.

draw stack on board

```
int fac(int n) {
  int k = n, f = 1;//<==
  while (k != 0) {
    f = mult(f, k);
    k = k-1;
  }
  return f;
}
```

```
fac:
    push {lr}        @ Save return addr
    sub sp, sp, #8   @ Allocate 8 bytes
    str r0, [sp,#4]  @ Save n as k
    movs r0, #1      @ Set f to 1
    str r0, [sp,#0]
    ...
```

Factorial Example: Accessing Local Variables

```
int fac(int n) {
    int k = n, f = 1;
    while (k != 0) {      ...
        f = mult(f, k);  ldr r0, [sp, #4]  @ fetch k
        k = k-1;  // <== subs r0, r0, #1   @ decrement it
    }                    str r0, [sp, #4]  @ save it again
    return f;            ...
}
```

Factorial Example: De-allocating Local Variables

```
int fac(int n) {
    int k = n, f = 1;
    while (k != 0) {
        f = mult(f, k);  ...
        k = k-1;         finish:
    }                        ldr r0, [sp, #0]
    return f;  // <==        add sp, sp, #8
}                            pop {pc}
```

Subroutine contract:

- Return value in `r0`
- Must leave `sp` unchanged (so stack is in same state as we found it)

?

How can we store (in memory) global variables?

Global Variables

Global variables are stored at a fixed absolute location. Remember why?

-
-

Global Variables

Global variables are stored at a fixed absolute location. Remember why?

Addresses are 32-bit quantities. How can we load the address into a register?

-
-

Global Variables

Global variables are stored at a fixed absolute location. Remember why?

Addresses are 32-bit quantities. How can we load the address into a register?

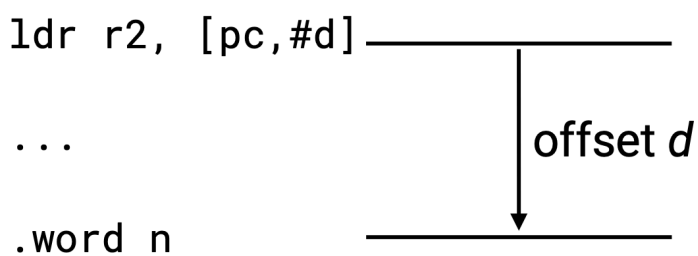
- Can use `movs rx, #255` to load 8 bits into a register.
-

Global Variables

Global variables are stored at a fixed absolute location. Remember why?

Addresses are 32-bit quantities. How can we load the address into a register?

- Can use `movs rx, #255` to load 8 bits into a register.
- Can store a 32-bit constant in memory, and load it.



Then can load *value* with `ldr r2, [r1, #0]`.

It’s annoying to manually manage addresses and offsets

`ldr` using `pc` as the base is again peculiar!

-

-

It's annoying to manually manage addresses and offsets

`ldr` using `pc` as the base is again peculiar!

- `pc` is always 4 ahead of instruction being executed…
  - Makes calculating offsets error-prone.
-

It's annoying to manually manage addresses and offsets

`ldr` using `pc` as the base is again peculiar!

- `pc` is always 4 ahead of instruction being executed…
  - Makes calculating offsets error-prone.
- Addresses need to be aligned so `addr % 4 == 0`.
  - Thumb instructions are 16-bit, so `pc` can be un-aligned.

It's annoying to manually manage addresses and offsets

`ldr` using `pc` as the base is again peculiar!

- `pc` is always 4 ahead of instruction being executed…
  - Makes calculating offsets error-prone.
- Addresses need to be aligned so `addr % 4 == 0`.
  - Thumb instructions are 16-bit, so `pc` can be un-aligned.

`ldr r2, [pc, #d]` rounds `pc` down to a multiple of 4 (how to implement this in hardware?) before adding offset.

It's annoying to manually manage addresses and offsets

`ldr` using `pc` as the base is again peculiar!

- `pc` is always 4 ahead of instruction being executed…
  - Makes calculating offsets error-prone.
- Addresses need to be aligned so `addr % 4 == 0`.
  - Thumb instructions are 16-bit, so `pc` can be un-aligned.

`ldr r2, [pc, #d]` rounds `pc` down to a multiple of 4 (how to implement this in hardware?) before adding offset.

⚠ **You don't want to memorise or deal with all this.**

Don't want to manually assign addresses, or remember peculiarities.

Relying on the Assembler

Can we get the assembler to?

- Select an address in RAM for the global variable?
- Store the address in ROM?
- Load the address from ROM without us needing to worry about the address of the address?

Global Variables: Example

```
int count = 0;
void increment(int n) {
  count = count + n;
}
```

```
   ldr r1, =count       @ Neat shorthand for ldr r2, [pc, #d]!
   ldr r2, [r1, #0]     @ r2: store value of count addr
   adds r0, r2, r0      @ r0: store n
   str r0, [r1, #0]
```

Now we must tell the assembler what `count` refers to.

Telling the Assembler about Locations in RAM

```
    .bss                    @ Place the following in RAM
    .balign 4               @ Align to a multiple of 4 bytes
count:
    .word 0                 @ Allocate a 4-byte word of memory
```

Assembler Input

```
    .text          @ I.e. ROM
    .thumb_func
func:
    ldr r1, =count
    ldr r2, [r1]
    adds r0, r2, r0
    str r0, [r1]
    bx lr
    .pool          @ Const pool
```

```
    .bss       @ I.e. RAM
    .align 4
count:
    .word 0
```

Assembler Output

```
Disassembly of section .text:
00000000 <func>:
   0:   4902            ldr     r1, [pc, #8]
   2:   680a            ldr     r2, [r1, #0]
   4:   1810            adds    r0, r2, r0
   6:   6008            str     r0, [r1, #0]
   8:   4770            bx      lr
   a:   0000            .short  0x0000
   c:   ????????        .word   <count>

Disassembly of section .bss:
00000000 <count>:
   0:   00000000        .word   0x00000000
```

Linker Output

```
Disassembly of section .text:
000003e4 <func>:
 3e4:   4902            ldr     r1, [pc, #8]
 3e6:   680a            ldr     r2, [r1, #0]
 3e8:   1810            adds    r0, r2, r0
 3ea:   6008            str     r0, [r1, #0]
 3ec:   4770            bx      lr
 3ee:   0000            .short  0x0000
 3f0:   20000020        .word   0x20000020

Disassembly of section .bss:
20000020 <count>:
20000020:  00000000     .word   0x00000000
```

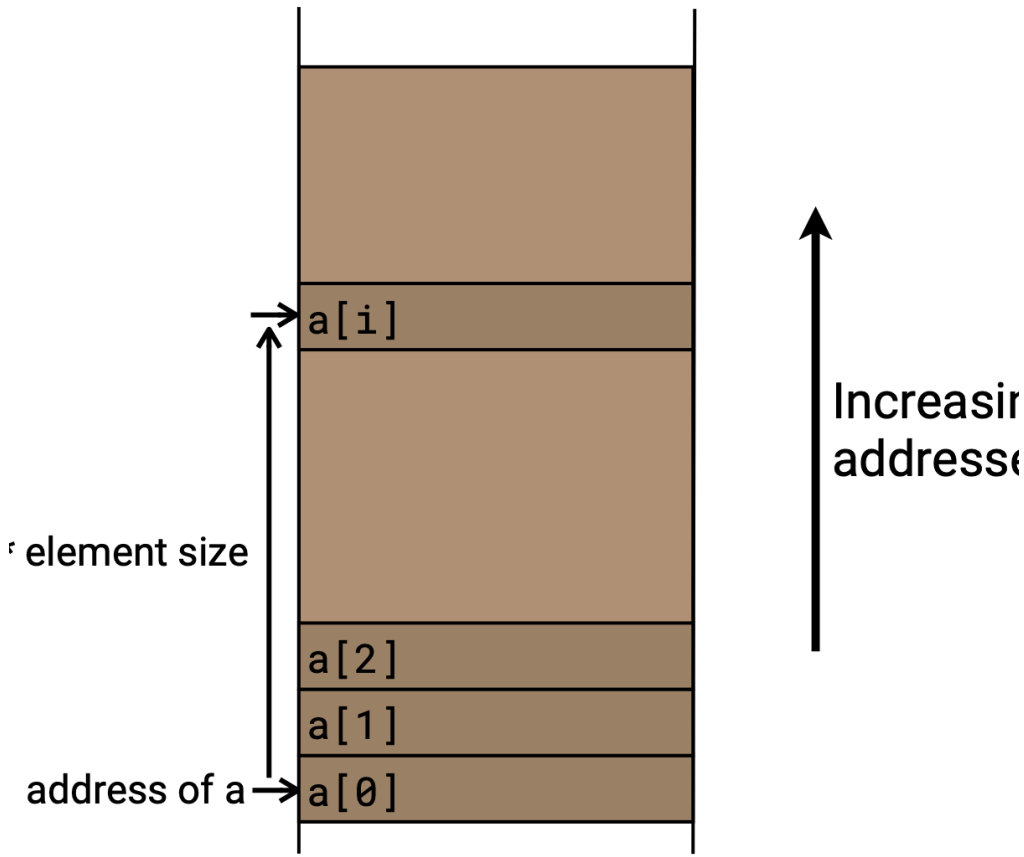
?

How can we store (in memory) arrays?

Array Addressing

- Arrays are just a collection of contiguous memory locations
- Can allocate arrays of fixed size in memory (`.bss`),
- ...or of variable size on stack.

```
static int account[10];
int deposit(int i, int a) {
  int x = account[i] + a;
  account[i] = x;
  return x;
}
```



Array Addressing in Assembly

```
deposit:
    ldr r3, =account  @ r3 is base of array
    lsls r2, r0, #2   @ r2 is 4*index
    ldr r0, [r3, r2]  @ Fetch balance
    adds r0, r0, r1   @ Add deposit
    str r0, [r3, r2]  @ Store back in array
    bx lr

    .bss
    .balign 4
account:
  .space 40           @ 40 bytes for 10 ints
```

Wrap-up

Other load/store instructions

`ldr` and `str` deal in 32-bit values, the size of a register. But there are also

- `ldrb` and `strb` for 8-bit values (useful for strings).
- `ldrh` and `strh` for 16-bit values.
- `ldrsb` and `ldrsh` to load 8- or 16-bit values with sign extension. Why is this necessary?

On Thumb, some of these exist only with the reg+reg addressing mode.

Summary

How can we access memory?

`ldr` and `str`.

How can we implement local variables?

Allocate on stack, and access with constant offsets.

How can we implement global variables?

Store address in ROM, load this, then access RAM location.

How can we implement arrays?

Access with addressing that adds two registers.

Mistake in the Last Lecture

About the behaviour of the `push` and `pop` instructions. Did anyone notice?

Mistake in the Last Lecture

About the behaviour of the `push` and `pop` instructions. Did anyone notice?

First person to e-mail me with what my mistake was gets a chocolate bar.

- You can find the result by looking something up in the datasheet, as we discussed.
- You will be able to write correct programs for the exam without knowing this detail.